Multilingual ticket routing

TensorForge 2.0 | MVP design, evaluation and findings
Team Hydra | Evidence reviewed 9 October 2026

Outcome. A locally trained classification pipeline, seven API operations and an integrated frontend are implemented. XLM-R leads the saved internal comparison at **99.83% primary accuracy (599/600)**. Final XLM-R evaluation on the supplied 800-ticket validation set is not present in the reviewed artifacts.

Problem and output contract

Route multilingual support tickets into 11 primary categories, an optional secondary category and a binary urgency flag. The API also returns a deterministic support-team mapping and primary confidence. It accepts chat, email and call-transcript channels, with an optional subject and ticket ID. English, Sinhala, Tamil and Latin-script transliterations must remain usable without language-specific stripping. [1, 2]

Design decisions and rationale

Decision	Reason and trade-off
Train local classifiers	TF-IDF models and fine-tuned multilingual encoders learn from labeled tickets. No external LLM is called for classification; training, validation and saved weights are inspectable.
Compare three model families	XLM-R, mDeBERTa and linear SVM test contextual representations against a cheaper sparse-text approach. Independent runs use the same splits for a fair comparison.
Shared encoder, three output heads	Neural models share language representations and learn primary, secondary and urgency outputs jointly. This reduces duplicate encoder inference but couples task optimization.
One API and frontend deployment	FastAPI serves both static UI and inference. Same-origin requests avoid a separate frontend service. SQLite persists jobs, with a deliberate single-process/single-replica constraint.

End-to-end path

Flow: ticket intake, authentication, schema validation, local inference, label and team routing, then JSON responses or persistent job results. The frontend exposes single-ticket routing, batch upload, job progress, cancellation and downloads. Keys remain in browser-tab memory and clear on refresh.

Model selection and deployment are separate: the API defaults to the bundled logistic-regression baseline unless a selected model is explicitly configured or baked into the Docker image. [2, 6]

Training and reproducibility

A shared evaluation protocol across three independent computers

Data and split discipline

The supplied synthetic dataset contains 4,000 training tickets and 800 validation tickets. Inputs concatenate channel, subject and body. Ticket IDs and language labels are excluded from model features; language is retained for analysis because the API does not provide it. Duplicate ticket IDs and exact duplicate model inputs are rejected. Unicode text and combining marks are preserved. [1, 3]

Partition	Rows	Use
Fit	2,800	Fit candidate configurations.
Tune	600	Choose hyperparameters and neural epoch.
Candidate refit	3,400	Retrain selected configuration on fit + tune.
Internal comparison	600	Rank independently trained candidate families.
Final confirmation	800	Supplied validation set; baseline evaluated, final XLM-R result not recorded.

The 600 comparison rows are held out from fitting and tuning, but become a model-selection set when choosing the winner. They are not an unbiased final test of that choice. Splits are primary-category stratified; seed 42, data hashes, split manifests and backbone revisions support reproducibility. Near-duplicate/template grouping is not implemented. [3, 4]

Models and selected parameters

Baseline: word TF-IDF (1-2 grams) plus character TF-IDF (3-5 grams), minimum document frequency 2, and 61,893 fitted features. Three logistic regressions use C=4, at most 1,000 iterations and seed 42; only urgency uses balanced class weights. This provides a compact, CPU-friendly reference. [2]

SVM: 18 primary settings combine three feature mixes, C in {0.25, 1, 4}, and class weighting on/off. Selected balanced word/character features use primary C=1 (unweighted), secondary C=4 (unweighted), urgency C=1 (balanced). Primary confidence uses three-fold sigmoid calibration within fitting data; the vocabulary is shared within those folds. [3, 4]

Neural candidates: XLM-R base and mDeBERTa-v3 base use masked mean pooling, dropout 0.1 and linear heads of 11 / 12 / 2 classes. Both saved configurations select learning rate 3e-5, five epochs, length 256, batch size 2 and accumulation 8 (effective batch 16). AdamW uses weight decay 0.01 excluding biases and normalization parameters, 10% warm-up, linear decay and gradient clipping at 1.0. [3, 4]

The objective is primary cross-entropy + 0.5 secondary loss + 0.5 urgency loss, with bounded inverse-frequency weights for the latter two. Mixed precision keeps trainable parameters in FP32 to avoid FP16 gradient-unscale failures. The intended hardware is three Ryzen 7 / 16 GB / RTX 4050 6 GB PCs, one candidate per PC; GPUs are not pooled. This is a hardware plan, not a measured cross-PC speed benchmark. [3]

Evaluation results

Different splits answer different questions; scores below must not be directly conflated

A. Baseline on supplied validation | n = 800

Target	Accuracy	Macro F1
Primary category	57.50%	0.6045
Secondary (including none)	94.25%	0.5412
Urgency	97.38%	0.9194
All three correct together	52.88%	-

The baseline was trained on all 4,000 training tickets. Validation was excluded from vocabulary fitting and parameter selection. Primary log loss was 1.5096; primary majority-class accuracy is 18%. Exact train/validation combined-input overlap was zero, which does not rule out semantic/template overlap. [2]

B. Candidate comparison | n = 600 internal training rows

Candidate	Primary acc.	Primary F1	Secondary acc.	Urgency acc.	Joint acc.
XLM-R	99.83%	0.9975	99.83%	99.83%	99.83%
Linear SVM	99.00%	0.9892	99.83%	99.83%	99.00%
mDeBERTa	98.83%	0.9860	99.00%	99.33%	97.50%

Ranking uses primary accuracy, then fixed-label primary macro F1, then joint accuracy. XLM-R has one primary error, SVM six and mDeBERTa seven. XLM-R exceeds SVM by 0.83 percentage points (five tickets). This small difference from a single split and seed does not establish statistical superiority. [4]

What the scores do and do not establish

Selected candidate: XLM-R. The 99.83% internal result supports choosing it for final confirmation. It does **not** demonstrate an increase from 57.50% to 99.83% on the official validation set: those values measure different models on different splits.

For XLM-R, secondary macro F1 is 0.4978 and urgency macro F1 is 0.9953. Secondary macro F1 averages all 12 fixed labels, including six with zero support and zero F1; therefore high accuracy and a lower macro F1 are compatible. Baseline macro F1 uses the labels present in truth/predictions, so the secondary macro-F1 definitions also differ. [2, 3, 4]

No finalized best_model artifact or winner validation report was found. The evidence supports an internal winner, not a final competition score. Reports and saved prediction arrays permit the comparison to be reproduced without retraining. [4]

Findings and error analysis

Aggregate accuracy hides rare-language, secondary-label and safety weaknesses

Language coverage of the selected XLM-R candidate

Internal comparison group	Tickets	Correct	Primary accuracy
English	214	214	100.00%
Sinhala	107	107	100.00%
Tamil	101	101	100.00%
Singlish	148	148	100.00%
Tanglish	11	10	90.91%
Mixed	19	19	100.00%

The only primary error is in Tanglish. The class report shows safety_conduct recall of 23/24 (95.83%) and one extra ride_trip_issue prediction; these counts identify a safety-to-trip misclassification. The 11-ticket Tanglish and 19-ticket mixed subsets are too small for strong generalization claims. [4]

Safety and secondary issues need targeted review

XLM-R: 59 of 60 urgent tickets were detected (98.33% recall), with no false-positive urgent predictions. Joint accuracy of 599/600 means the primary, secondary and urgency errors occur on the same ticket. Secondary labels are absent in 549/600 rows (91.5%); the secondary ride_trip_issue recall is 19/20. The high aggregate score must not obscure a missed urgent safety case. [4]

Baseline: urgency recall is 61/80 (76.25%), with 19 urgent tickets missed. Secondary accuracy is 94.25%, yet 46 of 80 actual secondary labels were missed; always predicting none already scores 90%. Primary food-quality and missing/wrong-order recall are weaknesses. These findings motivate contextual encoders and per-target evaluation rather than optimizing accuracy alone. [2]

Confidence and generalization limits

Neural confidence is an uncalibrated primary softmax probability; baseline confidence is logistic-regression probability. Neither has separate calibration evidence. The frontend displays confidence as an estimate, not a correctness guarantee. A review threshold should be selected from validation-based risk/coverage analysis, especially for safety and urgency.

All examples are synthetic. Sinhala, Tamil and transliterations were not independently reviewed by native speakers, and template overlap remains possible. Very high internal scores may reflect dataset regularities rather than robustness to unseen customer wording. No multi-seed study, template-held-out test, real-customer test or controlled ablation isolates why one candidate wins. [1, 3]

Next accuracy check: refit the selected configuration on 4,000 rows and evaluate once on the supplied 800 validation rows. If that set is used for further tuning, reserve a new untouched test set. Prioritize native-speaker review, template-grouped evaluation and urgent-case recall before autonomous triage.

Serving, verification and next steps

A working MVP with explicit deployment and evaluation boundaries

Operation	Behavior
GET /health	Public readiness and model version.
POST /predict	Single-ticket inference.
POST /predict/batch	Synchronous batch: 1-100 tickets.
POST /batch/jobs	Asynchronous batch: 1-5,000 tickets; idempotent submission.
GET /batch/jobs/{id}	Job state and progress.
GET /batch/jobs/{id}/results	Ordered, paginated predictions.
DELETE /batch/jobs/{id}	Cancel active work or discard a finished job.

Reliability: schema validation and authentication precede inference. X-API-Key and Bearer credentials use exact-key digest comparison; missing configuration fails closed. The key is loaded at startup without application expiry. SQLite and a persistent volume retain jobs; one running job and three queued jobs bound capacity. Interrupted work becomes failed/interrupted on restart. Non-root Docker packaging includes offline model loading. [5, 6]

Observed verification evidence

The latest API/frontend suite passed eight tests, with one optional neural test skipped; an earlier separate saved-XLM-R integration test passed. Eight neural predictions matched the tournament loader, including confidence within numerical tolerance. Real HTTP load checks returned all expected statuses: 96 authorized successes and 32 unauthorized responses at concurrency 16. Browser checks covered single inference, two-ticket synchronous batches and a 101-ticket background job with pagination. [5, 6]

Baseline development-host measurements: single-request p95 **9.91 ms** (30 requests), 100-ticket batch p95 **31.49 ms** (10 requests), startup **5.77 s**, and 5,000-ticket job acceptance **686 ms**. Acceptance time is not completion time. These are small synthetic workloads, not sustained load tests or neural-model latency results. [5]

Remaining release gates

1. Record final XLM-R validation and per-language/safety findings. **2.** Measure neural memory and latency on the competition 2-vCPU/4-GB target. **3.** Verify Docker execution and persistent-job recovery on the destination PC. **4.** Verify public HTTPS availability and exact-key authentication for the full judging window. Docker execution, reference-hardware performance and public hosting are not confirmed by the reviewed local evidence.

Evidence index

Paths relative to outputs/; saved runs supersede older README status statements.

[1] model-tournament/data/DATA_NOTES.md
[2] tensorforge/reports/evaluation.md; metrics.json; tensorforge/train.py
[3] model-tournament/computer_1_xlmr.py; computer_2_mdeberta.py; computer_3_linear_svm.py
[4] model-tournament/comparison/leaderboard.json; winner.json; results_*/result.json
[5] tensorforge/reports/hydra-verification.md; http-smoke.json
[6] tensorforge/reports/frontend-verification.md; app/; Dockerfile; TEAM_HYDRA.md